10 Python Case Study Solutions

Complete beginner-friendly programs based on the assigned Python syllabus

Topics used: variables, data types, operators, conditional statements, loops, strings, lists, tuples, sorting, and basic algorithmic thinking.

Note: The rates, thresholds, and concessions in these examples are sample values for demonstrating the logic. Replace them with the values specified by your teacher or case-study question. The programs use standard Python and do not require external libraries.

How to Use This PDF

Each case study includes a short algorithm and a complete Python program. Type the program into a Python editor and run it. Enter the requested values when prompted. The programs are educational examples, not official billing, scholarship, railway, medical, or financial policy systems.

Selected case studies

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Hospital Billing and Patient Classification
4. Employee Payroll and Deduction Analysis
5. Multi-Plan Telecommunications Cost Optimization
6. Railway Fare and Reservation Calculator
7. Retail Billing with Multiple Discount Rules
8. Manufacturing Cost and Profitability Analysis
9. Multi-Currency Travel Budget Analysis
10. Campus Event Financial Planning

The assignment lists 11 options; this PDF covers case studies 1–10.

1. University Scholarship Allocation

Algorithm

Read applicant details and subject marks. Check aggregate, every subject minimum, attendance, and family-income conditions. Keep eligible applicants in a list, then sort by aggregate (descending), attendance (descending), and income (ascending).

Python Code

```
# University Scholarship Allocation
# Sample policy values can be changed to match the question.
SUBJECT_MIN = 40
AGGREGATE_MIN = 75
ATTENDANCE_MIN = 85
INCOME_MAX = 250000

applicants = []
n = int(input("Number of applicants: "))

for i in range(n):
    print("\\nApplicant", i + 1)
    name = input("Name: ")
    marks = []
    for j in range(5):
        mark = float(input("Subject " + str(j + 1) + " mark (0-100): "))
        marks.append(mark)

    attendance = float(input("Attendance percentage: "))
    income = float(input("Annual family income: "))
    aggregate = sum(marks) / len(marks)

    subjects_pass = True
    for mark in marks:
        if mark < SUBJECT_MIN:
            subjects_pass = False

    if (subjects_pass and aggregate >= AGGREGATE_MIN
            and attendance >= ATTENDANCE_MIN and income <= INCOME_MAX):
        # Tuple: (name, aggregate, attendance, income)
        applicants.append((name, aggregate, attendance, income))

# Tie-break: aggregate high, attendance high, income low, then name A-Z.
applicants.sort(key=lambda x: (-x[1], -x[2], x[3], x[0].lower()))

print("\\nELIGIBLE SCHOLARSHIP RANKING")
if len(applicants) == 0:
    print("No eligible applicants.")
else:
    rank = 1
    for name, aggregate, attendance, income in applicants:
        print(rank, name, "| Average:", round(aggregate, 2),
              "| Attendance:", attendance, "% | Income:", income)
        rank += 1
```

2. Progressive Electricity Billing

Algorithm

For each month, calculate energy charges by consumption slabs, add fixed charge, apply surcharge, then subtract any qualifying rebate. Store monthly totals and flag an unusual increase compared with the previous month.

Python Code

```
# Progressive Electricity Billing
# Example slabs: first 100 units at 1.50, next 100 at 2.50,
# remaining units at 4.00. Replace rates with your assigned tariff.
FIXED_CHARGE = 75
SURCHARGE_RATE = 0.05
REBATE_LIMIT = 500
REBATE_RATE = 0.05
UNUSUAL_INCREASE_RATE = 0.30

def energy_charge(units):
    if units < 0:
        return -1
    charge = 0
    first = min(units, 100)
    charge += first * 1.50
    second = min(max(units - 100, 0), 100)
    charge += second * 2.50
    third = max(units - 200, 0)
    charge += third * 4.00
    return charge

months = []
bills = []
n = int(input("Number of months: "))

for i in range(n):
    month = input("Month name: ")
    units = float(input("Units consumed: "))
    if units < 0:
        print("Units cannot be negative. Try this month again.")
        continue

    energy = energy_charge(units)
    subtotal = energy + FIXED_CHARGE
    surcharge = subtotal * SURCHARGE_RATE
    total = subtotal + surcharge

    rebate = 0
    if total <= REBATE_LIMIT:
        rebate = total * REBATE_RATE
    total -= rebate

    months.append(month)
    bills.append(total)
    print(month, "| Energy:", round(energy, 2),
          "| Surcharge:", round(surcharge, 2),
          "| Rebate:", round(rebate, 2), "| Total:", round(total, 2))

print("\\nMONTH-TO-MONTH COMPARISON")
for i in range(len(bills)):
    if i == 0:
        print(months[i], "- no previous month for comparison")
    else:
        increase = bills[i] - bills[i - 1]
        if bills[i - 1] > 0:
            percent = increase / bills[i - 1] * 100
        else:
            percent = 0
        message = "UNUSUAL INCREASE" if percent > UNUSUAL_INCREASE_RATE * 100 else "Normal"
        print(months[i], "| Change:", round(increase, 2),
              "| Change %:", round(percent, 1), "|", message)
```

3. Hospital Billing and Patient Classification

Algorithm

Read consultation, diagnostic, and treatment charges. Reject negative charges. Apply a concession based on the supplied policy, calculate the final bill, and classify the patient using the payment amount.

Python Code

```
# Hospital Billing and Patient Classification
# Example policy: senior patients receive 10% off consultation;
# bills over 10,000 receive a 5% concession on the subtotal.
patients = []
n = int(input("Number of patients: "))

for i in range(n):
    print("\\nPatient", i + 1)
    name = input("Patient name: ")
    age = int(input("Age: "))
    consultation = float(input("Consultation charge: "))
    diagnostics = float(input("Diagnostic charges: "))
    treatment = float(input("Treatment charges: "))
    paid = float(input("Amount paid: "))

    if min(consultation, diagnostics, treatment, paid) < 0:
        print("Invalid: charges and payment cannot be negative.")
        continue

    subtotal = consultation + diagnostics + treatment
    concession = 0

    if age >= 60:
        concession += consultation * 0.10
    if subtotal > 10000:
        concession += subtotal * 0.05

    # A concession must not exceed the total bill.
    concession = min(concession, subtotal)
    final_bill = subtotal - concession
    balance = final_bill - paid

    if balance <= 0:
        status = "Paid in full"
    elif paid > 0:
        status = "Partially paid"
    else:
        status = "Unpaid"

    patients.append((name, subtotal, concession, final_bill, paid, balance, status))
    print("Subtotal:", round(subtotal, 2))
    print("Concession:", round(concession, 2))
    print("Final bill:", round(final_bill, 2))
    print("Balance (positive means due):", round(balance, 2))
    print("Classification:", status)

print("\\nPATIENT SUMMARY")
for record in patients:
    print(record[0], "| Final:", round(record[3], 2),
          "| Balance:", round(record[5], 2), "|", record[6])
```

4. Employee Payroll and Deduction Analysis

Algorithm

Calculate allowances, overtime pay, incentives, and gross salary. Apply progressive deductions to the relevant salary bands. Print a formatted payslip with gross pay, deductions, and net pay.

Python Code

```
# Employee Payroll and Deduction Analysis
# Example progressive deduction policy:
# 0% on first 15,000; 5% on next 10,000; 10% above 25,000.
employees = []
n = int(input("Number of employees: "))

for i in range(n):
    print("\\nEmployee", i + 1)
    name = input("Employee name: ")
    basic = float(input("Basic pay: "))
    hra_rate = float(input("HRA rate (%): "))
    other_allowances = float(input("Other allowances: "))
    overtime_hours = float(input("Overtime hours: "))
    overtime_rate = float(input("Overtime rate per hour: "))
    incentive = float(input("Incentive: "))

    if min(basic, hra_rate, other_allowances, overtime_hours,
           overtime_rate, incentive) < 0:
        print("Invalid negative value; employee skipped.")
        continue

    hra = basic * hra_rate / 100
    overtime_pay = overtime_hours * overtime_rate
    gross = basic + hra + other_allowances + overtime_pay + incentive

    band1 = min(gross, 15000)
    band2 = min(max(gross - 15000, 0), 10000)
    band3 = max(gross - 25000, 0)
    deduction = band1 * 0 + band2 * 0.05 + band3 * 0.10
    net = gross - deduction

    employees.append((name, basic, hra, other_allowances,
                      overtime_pay, incentive, gross, deduction, net))

    print("\\n--- SALARY STATEMENT ---")
    print("Employee:", name)
    print("Basic pay:       Rs.", format(basic, ".2f"))
    print("HRA:             Rs.", format(hra, ".2f"))
    print("Other allowance: Rs.", format(other_allowances, ".2f"))
    print("Overtime:        Rs.", format(overtime_pay, ".2f"))
    print("Incentive:       Rs.", format(incentive, ".2f"))
    print("Gross salary:    Rs.", format(gross, ".2f"))
    print("Deduction:       Rs.", format(deduction, ".2f"))
    print("Net salary:      Rs.", format(net, ".2f"))

print("\\nPAYROLL TOTALS")
total_gross = 0
total_deduction = 0
total_net = 0
for employee in employees:
    total_gross += employee[6]
    total_deduction += employee[7]
    total_net += employee[8]
print("Gross:", round(total_gross, 2),
      "| Deductions:", round(total_deduction, 2),
      "| Net:", round(total_net, 2))
```

5. Multi-Plan Telecommunications Cost Optimization

Algorithm

Read the user's data usage, required validity, and budget. For each plan, calculate base price plus excess data and validity charges. Keep plans that meet validity and budget, then choose the lowest total cost.

Python Code

```
# Multi-Plan Telecommunications Cost Optimization
# Plan tuple: (name, price, included_GB, validity_days, extra_GB_rate)
plans = [
    ("Basic", 199, 2, 28, 50),
    ("Smart", 299, 5, 28, 35),
    ("Max", 399, 10, 56, 25),
    ("Flex", 249, 4, 56, 40)
]

usage_gb = float(input("Expected data usage (GB): "))
required_days = int(input("Required validity (days): "))
budget = float(input("Budget (Rs.): "))

if usage_gb < 0 or required_days <= 0 or budget < 0:
    print("Invalid usage, validity, or budget.")
else:
    eligible = []
    print("\\nPLAN COMPARISON")
    for name, price, included_gb, validity, extra_rate in plans:
        excess_gb = max(usage_gb - included_gb, 0)
        excess_charge = excess_gb * extra_rate

        # If validity is insufficient, the plan is not eligible.
        total = price + excess_charge
        if validity >= required_days and total <= budget:
            eligible.append((total, name, price, excess_charge, validity))
        print(name, "| Base:", price, "| Excess charge:",
              round(excess_charge, 2), "| Validity:", validity,
              "| Total:", round(total, 2))

    if len(eligible) == 0:
        print("No plan satisfies both validity and budget.")
    else:
        eligible.sort(key=lambda x: (x[0], -x[4], x[1]))
        best = eligible[0]
        print("\\nMOST ECONOMICAL ELIGIBLE PLAN")
        print("Plan:", best[1])
        print("Total cost: Rs.", round(best[0], 2))
        print("Base price: Rs.", best[2])
        print("Excess charge: Rs.", round(best[3], 2))
        print("Validity:", best[4], "days")
```

6. Railway Fare and Reservation Calculator

Algorithm

For each passenger, calculate the distance-based base fare, apply the class multiplier and eligible age concession, then add the reservation fee. If cancelled, calculate the stated cancellation penalty and refund.

Python Code

```
# Railway Fare and Reservation Calculator
# Example policy rates: first 100 km Rs. 1.00/km, next 200 Rs. 0.80/km,
# distance above 300 km Rs. 0.60/km.
CLASS_MULTIPLIERS = (("Sleeper", 1.0), ("AC", 2.0), ("First", 3.0))
RESERVATION_FEE = 40

def base_fare(distance):
    first = min(distance, 100) * 1.00
    second = min(max(distance - 100, 0), 200) * 0.80
    third = max(distance - 300, 0) * 0.60
    return first + second + third

n = int(input("Number of passengers: "))
for i in range(n):
    print("\\nPassenger", i + 1)
    name = input("Name: ")
    age = int(input("Age: "))
    distance = float(input("Distance in km: "))
    travel_class = input("Class (Sleeper/AC/First): ").title()
    cancelled = input("Cancelled? (yes/no): ").lower()

    if age < 0 or distance <= 0:
        print("Invalid age or distance.")
        continue

    multiplier = 0
    for class_name, class_multiplier in CLASS_MULTIPLIERS:
        if travel_class == class_name:
            multiplier = class_multiplier

    if multiplier == 0:
        print("Unknown class; passenger skipped.")
        continue

    fare = base_fare(distance) * multiplier
    concession = 0
    if age < 5:
        concession = fare  # under-5 fare-free policy in this example
    elif age >= 60:
        concession = fare * 0.40

    total = fare - concession + RESERVATION_FEE

    if cancelled == "yes":
        penalty = min(total * 0.20, 200)
        refund = total - penalty
        print("Ticket total: Rs.", round(total, 2))
        print("Cancellation penalty: Rs.", round(penalty, 2))
        print("Refund: Rs.", round(refund, 2))
    else:
        print("Passenger:", name)
        print("Base fare after class multiplier: Rs.", round(fare, 2))
        print("Age concession: Rs.", round(concession, 2))
        print("Reservation fee: Rs.", RESERVATION_FEE)
        print("Final fare: Rs.", round(total, 2))
```

7. Retail Billing with Multiple Discount Rules

Algorithm

For every product, read price, quantity, and category. Calculate line totals, apply category-specific tax, compute threshold discount, cap the discount at the maximum concession, and print an itemized invoice.

Python Code

```
# Retail Billing with Multiple Discount Rules
# Category tax rates are examples; change to the assigned rates.
TAX_RATES = (("food", 0.05), ("clothing", 0.05),
             ("electronics", 0.18), ("other", 0.12))
DISCOUNT_THRESHOLD = 5000
DISCOUNT_RATE = 0.10
MAX_DISCOUNT = 1000

items = []
n = int(input("Number of product types: "))
subtotal = 0
tax_total = 0

for i in range(n):
    print("\\nProduct", i + 1)
    name = input("Product name: ")
    category = input("Category (food/clothing/electronics/other): ").lower()
    price = float(input("Unit price: "))
    quantity = int(input("Quantity: "))

    if price < 0 or quantity < 0:
        print("Invalid price or quantity; product skipped.")
        continue

    rate = 0.12
    for category_name, tax_rate in TAX_RATES:
        if category == category_name:
            rate = tax_rate

    line_total = price * quantity
    tax = line_total * rate
    subtotal += line_total
    tax_total += tax
    items.append((name, category, price, quantity, line_total, tax))

discount = 0
if subtotal >= DISCOUNT_THRESHOLD:
    discount = min(subtotal * DISCOUNT_RATE, MAX_DISCOUNT)

grand_total = subtotal - discount + tax_total

print("\\n================ INVOICE ================")
print("Product | Category | Qty | Amount | Tax")
for name, category, price, quantity, amount, tax in items:
    print(name, "|", category, "|", quantity, "|",
          round(amount, 2), "|", round(tax, 2))
print("-----------------------------------------")
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Tax total: Rs.", round(tax_total, 2))
print("GRAND TOTAL: Rs.", round(grand_total, 2))
```

8. Manufacturing Cost and Profitability Analysis

Algorithm

Calculate material requirement including wastage, material cost, labour cost, machine cost, total production cost, revenue, and profit. Compare profit with the target and display the result for each product batch.

Python Code

```
# Manufacturing Cost and Profitability Analysis
batches = []
n = int(input("Number of product batches: "))

for i in range(n):
    print("\\nBatch", i + 1)
    product = input("Product name: ")
    units = int(input("Units produced: "))
    material_per_unit = float(input("Material cost per unit: "))
    wastage_rate = float(input("Wastage percentage: "))
    labour_per_unit = float(input("Labour cost per unit: "))
    machine_cost = float(input("Total machine cost: "))
    selling_price = float(input("Selling price per unit: "))
    target_profit = float(input("Target profit (Rs.): "))

    if (units <= 0 or min(material_per_unit, wastage_rate,
            labour_per_unit, machine_cost, selling_price, target_profit) < 0):
        print("Invalid input; batch skipped.")
        continue

    material_base = units * material_per_unit
    wastage_cost = material_base * wastage_rate / 100
    material_total = material_base + wastage_cost
    labour_total = units * labour_per_unit
    total_cost = material_total + labour_total + machine_cost
    revenue = units * selling_price
    profit = revenue - total_cost
    margin = profit / revenue * 100 if revenue > 0 else 0

    status = "TARGET MET" if profit >= target_profit else "TARGET NOT MET"
    batches.append((product, total_cost, revenue, profit, margin, status))

    print("Material including wastage: Rs.", round(material_total, 2))
    print("Labour: Rs.", round(labour_total, 2))
    print("Machine: Rs.", round(machine_cost, 2))
    print("Total production cost: Rs.", round(total_cost, 2))
    print("Revenue: Rs.", round(revenue, 2))
    print("Profit/Loss: Rs.", round(profit, 2))
    print("Profit margin (%):", round(margin, 2))
    print(status)

print("\\nBATCH SUMMARY")
for product, cost, revenue, profit, margin, status in batches:
    print(product, "| Cost:", round(cost, 2), "| Revenue:",
          round(revenue, 2), "| Profit:", round(profit, 2), "|", status)
```

9. Multi-Currency Travel Budget Analysis

Algorithm

Read expense categories, amounts, currencies, exchange rates, and conversion fee rate. Convert each expense to the home currency, add fees, compare the total with the budget, and identify the largest category.

Python Code

```
# Multi-Currency Travel Budget Analysis
# Enter exchange rates as home-currency units for 1 unit of foreign currency.
rates = {
    "INR": 1.0,
    "USD": 83.0,
    "EUR": 90.0,
    "GBP": 105.0,
    "JPY": 0.56
}

fee_rate = float(input("Conversion fee percentage (e.g. 2 for 2%): "))
budget = float(input("Total travel budget in INR: "))
n = int(input("Number of expense categories: "))

expenses = []
total_spending = 0

for i in range(n):
    print("\\nExpense", i + 1)
    category = input("Category (food/hotel/transport/shopping/other): ")
    amount = float(input("Amount spent: "))
    currency = input("Currency (INR/USD/EUR/GBP/JPY): ").upper()

    if amount < 0:
        print("Negative expense is invalid; skipped.")
        continue
    if currency not in rates:
        print("Exchange rate unavailable; expense skipped.")
        continue

    converted = amount * rates[currency]
    fee = converted * fee_rate / 100
    total = converted + fee
    expenses.append((category, currency, amount, converted, fee, total))
    total_spending += total

print("\\nTRAVEL EXPENSE REPORT")
for category, currency, amount, converted, fee, total in expenses:
    print(category, "|", amount, currency, "| Converted: Rs.",
          round(converted, 2), "| Fee: Rs.", round(fee, 2),
          "| Total: Rs.", round(total, 2))

print("Total spending: Rs.", round(total_spending, 2))
print("Budget: Rs.", round(budget, 2))
difference = budget - total_spending
if difference >= 0:
    print("Within budget. Remaining: Rs.", round(difference, 2))
else:
    print("Over budget by Rs.", round(abs(difference), 2))

if len(expenses) > 0:
    largest = expenses[0]
    for expense in expenses:
        if expense[5] > largest[5]:
            largest = expense
    print("Largest expense category:", largest[0],
          "| Amount including fee: Rs.", round(largest[5], 2))
else:
    print("No valid expenses entered.")
```

10. Campus Event Financial Planning

Algorithm

Add fixed event costs and per-participant costs. Subtract sponsorship to find the amount to be funded by registrations. Calculate contribution per participant and determine the minimum number of registrations required to break even.

Python Code

```
# Campus Event Financial Planning
venue = float(input("Venue cost: "))
equipment = float(input("Equipment cost: "))
transport = float(input("Transport cost: "))
certificate_per_person = float(input("Certificate cost per participant: "))
catering_per_person = float(input("Catering cost per participant: "))
expected_participants = int(input("Expected participant count: "))
sponsorship = float(input("Confirmed sponsorship: "))
registration_fee = float(input("Registration fee per participant: "))

if min(venue, equipment, transport, certificate_per_person,
       catering_per_person, sponsorship, registration_fee) < 0:
    print("Costs, sponsorship, and fee cannot be negative.")
elif expected_participants < 0:
    print("Participant count cannot be negative.")
else:
    fixed_cost = venue + equipment + transport
    variable_per_person = certificate_per_person + catering_per_person
    expected_cost = fixed_cost + variable_per_person * expected_participants
    funding_needed = max(expected_cost - sponsorship, 0)
    contribution = registration_fee

    print("\\nEVENT BUDGET")
    print("Fixed costs: Rs.", round(fixed_cost, 2))
    print("Variable cost per participant: Rs.", round(variable_per_person, 2))
    print("Expected total cost: Rs.", round(expected_cost, 2))
    print("Sponsorship: Rs.", round(sponsorship, 2))
    print("Funding needed after sponsorship: Rs.", round(funding_needed, 2))

    # Break-even registrations: registration income + sponsorship
    # must cover fixed costs and per-person variable costs.
    net_contribution = registration_fee - variable_per_person
    remaining_fixed = max(fixed_cost - sponsorship, 0)

    if remaining_fixed == 0:
        minimum_registrations = 0
    elif net_contribution <= 0:
        minimum_registrations = None
    else:
        minimum_registrations = int(remaining_fixed / net_contribution)
        if minimum_registrations * net_contribution < remaining_fixed:
            minimum_registrations += 1

    if minimum_registrations is None:
        print("Break-even is impossible at this registration fee.")
    else:
        print("Minimum registrations to break even:",
              minimum_registrations)

    expected_income = sponsorship + registration_fee * expected_participants
    if expected_income >= expected_cost:
        print("Expected result: SURPLUS of Rs.",
              round(expected_income - expected_cost, 2))
    else:
        print("Expected result: SHORTFALL of Rs.",
              round(expected_cost - expected_income, 2))
```